# Simple Markov model

This reservoir architecture is a single delay whose input merges the token with its own permuted output. The rate_limit of 6.0 is relative to the input population of 10, so the limit is 60 elements. Because the limit acts on the merged input, the new token is subsampled together with the feedback.

In steady state, 10 fresh elements plus 60 fed back are cut to 60, so every element survives a step with probability 6/7. A token k steps back is therefore present as about 10·(6/7)^(k+1) elements, all carrying the permutation $\pi$^k.

So the state is one bundle of the recent tokens. Each token's position is coded by the depth of its permutation, and its recency by how many of its elements are left. Forgetting is stochastic and geometric, with a mean lifetime of 7 tokens; there are no boundaries and no explicit order k. The predictor stores every (state → next token) pair and matches by overlap. Whatever part of the history is discriminative carries the match, so the circuit behaves as a soft, variable-order Markov model. The pair counts are the evidence: on random strings over 26 letters, 2 to 3 tokens of context are already unique, and that is why it reaches 1.0.

## Requirements

```
<< "Circuits.m"
```

## Circuit

```
jsonstr =

{
  "$schema": "https://creatingintelligence.org/schemas/circuit-v1.json",

  "hyperparameters": {"default": [2000, 10]},

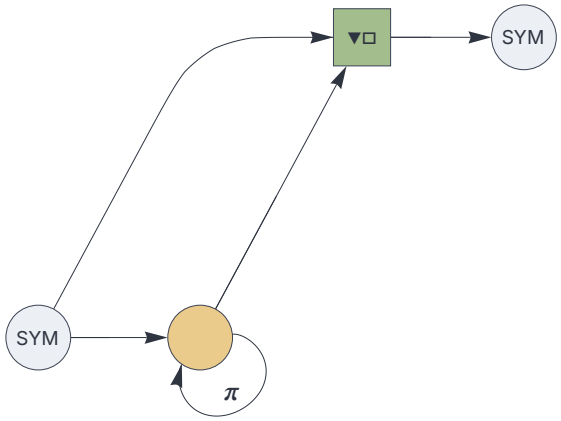
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
```

```
    {"component": "delay", "send": [51], "receive": [[11, {"slot": 51, "tags": ["permute"]}]],
"rate_limit": 6},

    {"component": "temporal", "send": [101], "receive": [11, 51]},

    {"component": "output", "plugin": "codec", "receive": [101]}
  ]
}
```

```
circ = Circuit[jsonstr];
f = circ["function"];
circ["schematics"][ImageSize→300]
```




## Dataset

Generate a 2000 character random string.

```
chars = CharacterRange["A", "Z"]
```

```
{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z}
```

```
dataset = RandomChoice[ chars, 2000];
```

## Prediction score

```
Do[
    predictions = f /@ dataset;
    Print["    iteration = ", i,
    ", prediction score = ", N[Mean@Boole@MapThread[SameQ,
        {dataset, RotateRight[predictions]}]]],
    {i, 1, 5}];
```

```
iteration = 1, prediction score = 0.033
iteration = 2, prediction score = 0.9785
iteration = 3, prediction score = 0.9985
iteration = 4, prediction score = 1.
iteration = 5, prediction score = 1.
```